— Solution notes —

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 5

5 Concurrent and Distributed Systems (tlh20)

A system is being designed to measure traffic flow into a city. Part of this system is a set of monitoring nodes, each using sensors to detect when a vehicle passes the monitoring node, and incrementing a per-node counter. The nodes communicate over a network to provide a city-wide total.

System models and faults

(*a*) Define each of the terms:

(*i*) Fair-loss network links. [1 mark]

(*ii*) Crash-recovery execution. [1 mark]

(*iii*) Asynchronous timing. [1 mark]

Answer: Numbers in square brackets refer to key slides or sections in the 2022-2023 course. This first part covers standard definitions from the course:

- A fair-loss link allows messages to be lost, duplicated, or re-ordered. If kept retrying, a message eventually gets through. [33]
- Under crash-recovery, a node may crash at any moment, losing its in-memory state. It may resume execution sometime later. Data stored on disk survives the crash. [34]
- Asynchronous timing means that messages can be delayed arbitrarily, and that nodes can pause execution arbitrarily. [35]

Replication; quorums

(*b*) Consider a version of the system using *quorum-based replication*. There is a fixed set of 5 nodes, meaning that each node holds a 5-tuple comprising the node's most recent values for each of the replicated counters. A node should be able to operate if it can communicate with at least 2 other nodes. Describe how each of the following operations can be implemented by sending messages between nodes:

(*i*) A `setCount(n)` function to update the node's local counter to `n` and to replicate the change to a quorum. [5 marks]

(*ii*) A `getTotalCount()` function to return the total of the counters from all of the nodes. [5 marks]

You should describe the messages sent and received by each node, along with how a node updates its local 5-tuple with new information when it receives messages, and how a node determines that the operation is complete.

Answer: Along with the final section of this question, this part relates to the CAP theorem that a system can either provide strong consistency or availability in the presence of network partitions. [139]

The solution notes below use pseudo-code to set out one approach in detail; this is not expected for full marks. Some specific points to watch for in the algorithm are:

- Updates in `setCount` must be replicated in order to allow `getTotalCount` to complete without all nodes available.
- Asynchronous communication means that messages may be duplicated. Hence we should track sets of responders, rather than counts of response messages.
- An update request message may be delayed, hence we combine the incoming state with the local state via `max` rather than assuming the incoming state is fresh.
- Similarly, responses to earlier requests may be delayed or duplicated. Hence the use of `request_seq` to ensure that delayed responses to an old request are not re-used as fresh.

```
// State at each node:
int[] counters = [0, 0, 0, 0, 0]
int request_seq = 0

// Message types:
updateCounterReq(int sender, int value)
updateCounterResp(int responder, int value)
getCounterReq(int sender, int sender_seq)
getCounterResp(int responder, int sender_seq, int[] values)

on receiving updateCounterReq(m_sender, m_value):
  counters[m_sender] = max(counters[m_sender], m_value)
  send to m_sender updateCounterResp(my_id, m_value)

on receiving getCounterReq(m_sender, m_sender_seq):
  send to m_sender getCounterResp(my_id, m_sender_seq, counters)

void setCount(int value):
  counters[my_id] = value
  responders = {my_id}
retry: // Repeat until we get a quorum
  try:
    for each node in other_nodes: // Send requests to other nodes
      send to node updateCounterReq(my_id, value)
    while (|responders| < 3):
      (m_responder, m_value) = receive updateCounterResp
      if m_value == value // Response is for this request
        responders = responders + {m_responder}
  catch timeout:
    goto retry // Send requests again, continue accumulating responders

int getTotalCount():
  request_seq ++
  responders = {my_id}
retry: // Repeat until we get a quorum
  try:
    for each node in other_nodes: // Send requests to other nodes
      send to node getCounterReq(my_id, request_seq)
    while (|responders| < 3):
      (m_responder, m_sender_seq, m_values) = receive getCounterResponse
      if m_sender_seq == request_seq // Response is for this request
        responders = responders + {m_responder}
        counters = max(counters, m_values)
  catch timeout:
    goto retry // Send requests again, continue accumulating responders
```

```
  return sum(counters[0..5])
}
```

Replica consistency; linearizability

(*c*) Does your system provide *linearizable* behaviour? Either explain why your system is linearizable, or provide an example showing a non-linearizable result. [3 marks]

Answer: The pseudo-code above is not linearizable. As an example, suppose that a `setCount` operation has partially executed and updated only 2 nodes (including itself). Next, a `getTotalCount` runs, reads from a quorum including these nodes and returns a result including the `setCount`. Finally, another `getTotalCount` runs, reads from a quorum *excluding* the update and returns a result omitting the `setCount`. The two reads are not linearizable because they do not reflect their real-time ordering.

Note that either answer is possible depending on the pseudo-code provided. My expectation is that most answers will *not* be linearizable and will exhibit the same kind of behavior summarized above and in the lectures [138]. Linearizability could be provided by a variant of the ABD algorithm [143] with a `getTotalCount` ensuring that a quorum of nodes holds the value computed before returning.

Replica consistency; eventual consistency

(*d*) Consider a second version of the system that provides *strong eventual consistency* and allows the operations to always complete irrespective of the number of nodes available for communication. Summarize the changes needed to provide this behaviour. [4 marks]

Answer: To run to completion irrespective of the number of nodes available, we must avoid any cases where a node will loop indefinitely based on timeouts or other reasons. In the pseudo-code above, this means removing the loops and allowing a `setCount` operation complete after only partially replicating its value (perhaps only to itself), and allowing a `getTotalCount` to use stale values if insufficient responses have been received.

In addition, strong eventual consistency requires eventual delivery of updates and convergence between nodes [149]. This is not achieved by the pseudo-code above because updates are only disseminated during `setCount` and there is no guarantee that a node will continue to call that function. One approach is to add a periodic repetition of `setCount` even when there is no change to the value. Another approach is to add a gossip mechanism between nodes to distribute one another's values.